

Likewise, if the topology of an existing deployment needs to change in order to, for example, scale to support greater load, the application logic can remain the same.

Rich with Features...

MongoDB is a general-purpose database, so aside from creating, reading, updating, and deleting data, it provides most of the features you would expect from a database management system and many others that set it apart. These include:

Indexing

MongoDB supports generic secondary indexes and provides unique, compound, geospatial, and full-text indexing capabilities as well. Secondary indexes on hierarchical structures such as nested documents and arrays are also supported and enable developers to take full advantage of the ability to model in ways that best suit their applications.

Aggregation

MongoDB provides an aggregation framework based on the concept of data processing pipelines. Aggregation pipelines allow you to build complex analytics engines by processing data through a series of relatively simple stages on the server side, taking full advantage of database optimizations.

Special collection and index types

MongoDB supports time-to-live (TTL) collections for data that should expire at a certain time, such as sessions and fixed-size (capped) collections, for holding recent data, such as logs. MongoDB also supports partial indexes limited to only those documents matching a criteria filter in order to increase efficiency and reduce the amount of storage space required.

File storage

MongoDB supports an easy-to-use protocol for storing large files and file metadata.

Some features common to relational databases are not present in MongoDB, notably complex joins. MongoDB supports joins in a very limited way through use of the `$lookup` aggregation operator introduced in the 3.2 release. In the 3.6 release, more complex joins are possible using multiple join conditions as well as unrelated subqueries. MongoDB's treatment of joins were architectural decisions to allow for greater scalability, because both of those features are difficult to provide efficiently in a distributed system.

...Without Sacrificing Speed

Performance is a driving objective for MongoDB, and has shaped much of its design. It uses opportunistic locking in its WiredTiger storage engine to maximize concurrency and throughput. It uses as much RAM as it can as its cache and attempts to automatically choose the correct indexes for queries. In short, almost every aspect of MongoDB was designed to maintain high performance.

Although MongoDB is powerful, incorporating many features from relational systems, it is not intended to do everything that a relational database does. For some functionality, the database server offloads processing and logic to the client side (handled either by the drivers or by a user's application code). Its maintenance of this streamlined design is one of the reasons MongoDB can achieve such high performance.

The Philosophy

Throughout this book, we will take the time to note the reasoning or motivation behind particular decisions made in the development of MongoDB. Through those notes we hope to share the philosophy behind MongoDB. The best way to summarize the MongoDB project, however, is by referencing its main focus—to create a full-featured data store that is scalable, flexible, and fast.

Getting Started

MongoDB is powerful but easy to get started with. In this chapter we'll introduce some of the basic concepts of MongoDB:

- A *document* is the basic unit of data for MongoDB and is roughly equivalent to a row in a relational database management system (but much more expressive).
- Similarly, a *collection* can be thought of as a table with a dynamic schema.
- A single instance of MongoDB can host multiple independent *databases*, each of which contains its own collections.
- Every document has a special key, "_id", that is unique within a collection.
- MongoDB is distributed with a simple but powerful tool called the *mongo shell*. The *mongo* shell provides built-in support for administering MongoDB instances and manipulating data using the MongoDB query language. It is also a fully functional JavaScript interpreter that enables users to create and load their own scripts for a variety of purposes.

Documents

At the heart of MongoDB is the *document*: an ordered set of keys with associated values. The representation of a document varies by programming language, but most languages have a data structure that is a natural fit, such as a map, hash, or dictionary. In JavaScript, for example, documents are represented as objects:

```
{"greeting" : "Hello, world!"}
```

This simple document contains a single key, "greeting", with a value of "Hello, world!". Most documents will be more complex than this simple one and often will contain multiple key/value pairs:

```
{"greeting" : "Hello, world!", "views" : 3}
```

As you can see, values in documents are not just “blobs.” They can be one of several different data types (or even an entire embedded document—see “[Embedded Documents](#)” on page 19). In this example the value for “greeting” is a string, whereas the value for “views” is an integer.

The keys in a document are strings. Any UTF-8 character is allowed in a key, with a few notable exceptions:

- Keys must not contain the character `\0` (the null character). This character is used to signify the end of a key.
- The `.` and `$` characters have some special properties and should be used only in certain circumstances, as described in later chapters. In general, they should be considered reserved, and drivers will complain if they are used inappropriately.

MongoDB is type-sensitive and case-sensitive. For example, these documents are distinct:

```
{"count" : 5}  
{"count" : "5"}
```

as are these:

```
{"count" : 5}  
{"Count" : 5}
```

A final important thing to note is that documents in MongoDB cannot contain duplicate keys. For example, the following is not a legal document:

```
{"greeting" : "Hello, world!", "greeting" : "Hello, MongoDB!"}
```

Collections

A *collection* is a group of documents. If a document is the MongoDB analog of a row in a relational database, then a collection can be thought of as the analog to a table.

Dynamic Schemas

Collections have *dynamic schemas*. This means that the documents within a single collection can have any number of different “shapes.” For example, both of the following documents could be stored in a single collection:

```
{"greeting" : "Hello, world!", "views": 3}  
{"signoff": "Good night, and good luck"}
```

Note that the previous documents have different keys, different numbers of keys, and values of different types. Because any document can be put into any collection, the question often arises: “Why do we need separate collections at all?” With no need for

separate schemas for different kinds of documents, why *should* we use more than one collection? There are several good reasons:

- Keeping different kinds of documents in the same collection can be a nightmare for developers and admins. Developers need to make sure that each query is only returning documents adhering to a particular schema or that the application code performing a query can handle documents of different shapes. If we're querying for blog posts, it's a hassle to weed out documents containing author data.
- It's much faster to get a list of collections than to extract a list of the types of documents in a collection. For example, if we had a "type" field in each document that specified whether the document was a "skim," "whole," or "chunky monkey," it would be much slower to find those three values in a single collection than to have three separate collections and query the correct collection.
- Grouping documents of the same kind together in the same collection allows for data locality. Getting several blog posts from a collection containing only posts will likely require fewer disk seeks than getting the same posts from a collection containing posts and author data.
- We begin to impose some structure on our documents when we create indexes. (This is especially true in the case of unique indexes.) These indexes are defined per collection. By putting only documents of a single type into the same collection, we can index our collections more efficiently.

There are sound reasons for creating a schema and for grouping related types of documents together. While not required by default, defining schemas for your application is good practice and can be enforced through the use of MongoDB's documentation validation functionality and object-document mapping libraries available for many programming languages.

Naming

A collection is identified by its name. Collection names can be any UTF-8 string, with a few restrictions:

- The empty string ("") is not a valid collection name.
- Collection names may not contain the character `\0` (the null character), because this delineates the end of a collection name.
- You should not create any collections with names that start with *system.*, a prefix reserved for internal collections. For example, the *system.users* collection contains the database's users, and the *system.namespaces* collection contains information about all of the database's collections.